



SIT314

Software Architecture and Scalability for Internet-
Of-Things

Learning Summary Report

Hayes Duong
222610226

Self-Assessment Details

The following checklists provide an overview of my self-assessment for this unit.

	Pass (D)	Credit (C)	Distinction (B)	High Distinction (A)
Self-Assessment				✓

Self-Assessment Statement

Declaration

I declare that this portfolio is my individual work. I have not copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part of this submission been written for me by another person.

Signature: **Hayes Duong**

Portfolio Overview

This portfolio includes work that demonstrates that I have achieved all Unit Learning Outcomes for SIT314 – Software Architecture and Scalability for IoT to a **High Distinction** level.

A - Learning journey

Start - I entered the unit with working knowledge of AWS and Node.js but without a robust, end-to-end method for architecting and validating large-scale, cloud-backed IoT systems.

Build & prove - I delivered a production-style event-driven smart-lighting system on AWS IoT Core with a Node-RED decision layer behind ALB + Auto Scaling Group and a serverless actuation/data plane (API Gateway → Lambda → DynamoDB). Crucially, I implemented state-change detection and validated elastic scale-out/scale-in under thousands of MQTT/TLS clients — establishing clear scalability evidence and operational discipline.

Extend beyond the unit - I then re-architected the solution as four microservices (API Gateway, Sensor Data Service, Automation Service, Light Control Service) based on unit chair feedback upon previous architecture (task 6.3D) and deployed them on Kubernetes with Horizontal Pod Autoscaling (HPA). Under ~1,400 simulated devices, the system scaled from 5 to 24 pods, with the Sensor Data Service growing from 1 → 15 replicas, and I analysed bottlenecks (AWS IoT Core connection limits, I/O-bound ingestion, DB constraints) and proposed concrete optimisations (batch writes, custom/event metrics via KEDA, tracing) (task 6.4HD).

B – Why this merits High Distinction

Beyond-unit extension - I moved from serverless/ASG elasticity to Kubernetes microservices with HPA, tuned per workload (CPU/memory targets, stabilisation windows), and evaluated behaviour under realistic fan-in — work that explicitly aligns with the unit's HD brief to apply state-of-the-art scalability research in a designed and evaluated system.

Measured scalability - Evidence includes autoscaling traces and counts (e.g., Sensor Data Service 1→15 pods at peak) and throughput tallies under 500–1,400 devices, not just diagrams — demonstrating real horizontal scale, headroom analysis, and controlled failure modes.

Architectural rigor - I documented service responsibilities, data models, security (secrets, IAM/certs), health probes, and state-change suppression that cuts unnecessary writes/commands — turning principles into measurable outcomes.

Critical evaluation - I identified and isolated external bottlenecks (managed MQTT quotas), separated I/O-bound vs CPU-bound behaviour, and proposed batching, custom metrics/KEDA, and tracing as next steps — showing maturity in diagnosis and optimisation planning.

C - ULO mapping

ULO1 — Understand key challenges & principles of large-scale, cloud-based CPS

- I addressed device churn, variable load, back-pressure, idempotency, and quota limits; designed for fault isolation and observability; and reasoned about I/O-bound ingestion vs CPU-bound automation.
- Evidence: workload characterization and bottleneck analysis in my Kubernetes report.

ULO2 — Select, justify, and apply architecture & patterns to functional + NFRs

- I justified event-driven ingestion, state-change suppression to reduce writes, stateless computing behind ALB/ASG, then microservices + HPA with per-service thresholds and stabilization to meet latency/cost SLOs.
- Design choices are tied to measured results and ADR-style rationale across both submissions.

ULO3 — Develop large-scale CPS using available services

- I built working systems that ingest, decide, actuate, and persist at scale: AWS IoT → Node-RED/ALB/ASG → API GW/Lambda/DynamoDB; then Kubernetes-based microservices with MongoDB time-series storage and autoscaling.
- The artefacts, load scripts, and database evidence are included in my portfolio submission.

ULO4 — Deploy and scale cloud-based services

- I demonstrated autoscaling on AWS (ASG) and on Kubernetes (HPA), with health probes, secrets, and environment-specific configs, and I reported replica counts, CPU targets, and scaling behavior under 500–1,400 devices — fulfilling the unit's requirement to deploy and demonstrate automatic scaling.

Reflections

The most important things I learnt:

Every introduced concept, material in this unit is equally important to me because they taught me about the scalability, commonly-use software architecture in tech-industry / enterprise which are part of essential knowledge that every software engineer / developer should know beside coding because these will determine how a final product will be in term of architecture, and how will this product reaching the hand of end users in terms of scalability.

I feel I learnt these topics, concepts, and/or tools well:

I am confident with the introduced topics from the unit by now because they are straightforward, practical and can easily being “visualized” as one doing it.

I found the following topics particularly challenging:

Despite saying that I am confident with everything, Kubernetes is the concept that I am still afraid of (despite our unit not ventured too far with it) because I am still struggling with composing a good .yaml file to deploy microservice pod.

I found the following topics particularly interesting:

I found it lucky for me that the unit re-introduced me to AWS and its service and requested students to work with it using our developed program. This helped me solidify my existing knowledge on AWS from SIT233 – Cloud Computing. To be honest, if I did not enroll for this unit, I would not be confident with AWS services because the previous unit was lecture heavily which completely overwhelmed me.

I still need to work on the following areas:

AWS, Docker & Kubernetes are the areas that I need to work more even after completing this unit – because of its complexity in configuration. One may well-understand their concepts but will have a difficult time with them if he or she neglects it – just like coding.

The things that helped me most were:

Things that helped me the most were feedbacks from my tutor and unit chair for guiding me throughout this unit without too astray from the unit expectations (reflected through my task 6.3D where I did not follow the expectation of developing a microservice architecture for my project; however, I managed to switch to microservice architecture in task 6.4HD).

My progress in this unit was ...:



So far, I can tell that I am right on time for 99% of my assignments with the exception of Learning Summary (12.1P).

If I did this unit again, I would do the following things differently:

If I did this unit again, I would pick an interesting topic for task 6.3D and approach task 6.4HD with a newer method or extend it with GKE, or Kubernetes service provided by AWS.

In addition, I would try to get both D and HD submitted on time rather than request for extension.

Other...:

SIT314 has been the pivot point where “writing code” matured into thinking in systems. Working through cloud-backed, event-driven IoT architectures forced me to internalise how non-functional requirements—latency, reliability, security, cost—drive every design choice, and to justify those choices with data.

I learned to translate vague product goals into clear architectures, document decisions (ADRs), instrument services with metrics and traces, and validate scales through automation and load testing. Just as importantly, I practiced the habits I’ll need as a professional engineer and future tech lead: communicating trade-offs plainly, designing for operability (IaC, runbooks, SLOs), building with least-privilege security, and mentoring through code reviews and design notes. This unit has shaped my career path in two ways.

Near-term, I’m ready to contribute as a software engineer who can own a service end-to-end—requirements, design, build, deploy, and observe. Long-term, it has given me architectural thinking, cost-aware mindset, and evidence-based decision style that underpin effective technical leadership.

I leave the unit with a toolkit—and a discipline—for designing scalable, secure, and maintainable systems, and a clear roadmap toward becoming a software architect and, ultimately, a tech lead.